

EXPERMENT: Decision Tree.

Aim

To implement a Decision Tree classification algorithm in Python and classify data based on selected features.

Objectives

1. To understand the Decision Tree classification technique.
2. To construct a tree using feature-based decisions.
3. To classify unknown data.
4. To understand concepts such as root nodes, branches and leaf nodes.
5. To evaluate the classification result.

Algorithm

1. Load or define the training dataset.
2. Separate the input features and target class.
3. Select the best feature for splitting the data.
4. Create a decision node for the selected feature.
5. Divide the dataset into subsets.
6. Repeat the splitting process recursively.
7. Stop when the data belongs to a single class or the stopping condition is reached.
8. Assign the class label to the leaf node.
9. Use the constructed tree to predict new data.
10. Display the prediction and accuracy.

Code

```
import math

data = [
    ['Sunny', 'Hot', 'No'],
    ['Sunny', 'Cool', 'Yes'],
    ['Rainy', 'Cool', 'Yes'],
    ['Rainy', 'Hot', 'No'],
    ['Cloudy', 'Cool', 'Yes'],
    ['Cloudy', 'Hot', 'Yes']
]
```

```

features = ['Weather', 'Temperature']

def entropy(rows):
    total = len(rows)
    counts = {}
    for row in rows:
        label = row[-1]
        counts[label] = counts.get(label, 0) + 1
    result = 0
    for count in counts.values():
        probability = count / total
        result -= probability * math.log2(probability)
    return result

def information_gain(rows, column):
    total_entropy = entropy(rows)
    values = set(row[column] for row in rows)

    weighted_entropy = 0
    for value in values:
        subset = [row for row in rows if row[column] == value]
        weighted_entropy += (
            len(subset) / len(rows)
        ) * entropy(subset)
    return total_entropy - weighted_entropy

def majority_class(rows):
    counts = {}
    for row in rows:
        label = row[-1]
        counts[label] = counts.get(label, 0) + 1
    return max(counts, key=counts.get)

def build_tree(rows, columns):
    labels = [row[-1] for row in rows]

```

```

if len(set(labels)) == 1:
    return labels[0]
if not columns:
    return majority_class(rows)
best_column = max(
    columns,
    key=lambda c: information_gain(rows, c)
)
tree = {features[best_column]: {}}
values = set(row[best_column] for row in rows)
remaining_columns = [
    c for c in columns if c != best_column
]
for value in values:
    subset = [
        row for row in rows
        if row[best_column] == value
    ]
    tree[features[best_column]][value] = build_tree(
        subset,
        remaining_columns
    )
return tree

tree = build_tree(data, [0, 1])
print("Decision Tree:")
print(tree)
test = ['Cloudy', 'Hot']
node = tree
while isinstance(node, dict):
    feature = list(node.keys())[0]
    value = test[features.index(feature)]

```

```
node = node[feature][value]

print("Test Data:", test)

print("Predicted Class:", node)
```

Output



```
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:/Users/shaik arshad basha/OneDrive/Desktop/Map Coloring using CSP.py
Decision Tree:
{'Temperature': {'Cool': 'Yes', 'Hot': {'Weather': {'Rainy': 'No', 'Cloudy': 'Yes', 'Sunny': 'No'}}}}
Test Data: ['Cloudy', 'Hot']
Predicted Class: Yes
>>> |
```

Result

The Decision Tree was successfully constructed using entropy and information gain, and the test sample was correctly classified.

Conclusion

The experiment demonstrates how a Decision Tree makes classification decisions by recursively selecting the most informative features. It is simple to understand and useful for many classification problems.